Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

Звіт

про виконання лабораторної роботи №4

“Професійне ООП та система типів Python”

Виконав:
студент 3 курсу
групи ФЕП-33с
Качмар Денис

Викладач:
Проф. Свелеба Сергій Андрійович

Львів - 2026

ХІД РОБОТИ

Тема: Професійне ООП та система типів Python
Мета: Сформувати практичні навички побудови професійної об’єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів (ABC), протоколів (Protocol), датакласів (dataclass), властивостей (properties), спеціальних dunder-методів, принципів SOLID та сучасної системи статичної типізації (type hints, TypeVar, Generic, TypedDict) з верифікацією через mypy у строгому режимі.
Варіант: 5 (Каталог автомобілів)

Постановка задачі:
Спроєктувати та реалізувати типізовану об’єктно-орієнтовану модель предметної області «Каталог автомобілів» відповідно до Варіанта 5: 1. Виділити базові domain entities: **`Vehicle`**, **`Car`**, **`ElectricCar`**, **`VehicleCatalog`**, **`VehicleSpecification`**. 2. Реалізувати імутабельні Value Objects: **`Money`**, **`Mileage`**, **`VIN`** на базі **`@dataclass(frozen=True, slots=True)`** із валідацією стану та арифметичними dunder-методами. 3. Реалізувати наслідування (**`Vehicle`** -> **`Car`** -> **`ElectricCar`**) із викликом **`super()`** та перевизначенням поведінки. 4. Застосувати композицію (has-a) замість надлишкового наслідування (**`VehicleSpecification`** у складі **`Vehicle`**). 5. Забезпечити інкапсуляцію через приватні атрибути та контрольовані властивості (**`@property`** і **`@setter`**) із валідацією неприпустимих значень (ціна < 0, пробіг < 0). 6. Описати обов’язкові інтерфейси за допомогою **`abc.ABC`** (**`DiscountPolicy`**, **`AbstractSearchCriteria`**, **`Repository[T]`**) та структурні інтерфейси за допомогою **`typing.Protocol`** з **`@runtime_checkable`** (**`SearchCriteria`**, **`VehicleNotifier`**, **`HasId`**, **`Priceable`**, **`Exportable`**). 7. Реалізувати dunder-методи: **`__len__`**, **`__iter__`**, **`__getitem__`**, **`__contains__`**, **`__add__`**, **`__eq__`**, **`__lt__`**, **`__hash__`**, **`__str__`**, **`__repr__`**. 8. Побудувати параметризований Generic Repository (**`InMemoryRepository[T]`** із **`T = TypeVar("T", bound=HasId)`**). 9. Спроєктувати шар Application Service (**`CatalogService`**) із застосуванням Dependency Injection (передача абстракцій **`Repository`**, **`VehicleNotifier`**, **`DiscountPolicy`**). 10. Реалізувати DTO та роботу з зовнішніми структурами даних через **`TypedDict`** (**`VehiclePayload`**, **`ElectricVehiclePayload`**) і generic-контейнер **`Result[T]`**. 11. Продемонструвати застосування всіх 5 принципів SOLID (SRP, OCP, LSP, ISP, DIP). 12. Провести обов’язковий експеримент статичної перевірки типів (Section 71) із навмисним внесенням трьох помилок типізації та їх виправленням у mypy. 13. Провести дослідження двох варіантів проєктування: жорстке зв’язування проти Dependency Injection (Section 72), а також порівняння наслідування та композиції (Section 73). 14. Налаштувати строгий статичний аналізатор **`mypy`** (**`strict = true`**) у **`pyproject.toml`** та забезпечити повну сумісність із попередніми лабораторними роботами №1–№3.

---

1. Опис предметної області (Варіант 5: Каталог автомобілів)

Предметна область охоплює облік, каталогізацію, фільтрацію, аналітику та бізнес-обробку транспортних засобів у дилерському центрі або торговельному майданчику. Система оперує як класичними автомобілями з двигунами внутрішнього згоряння (ICE), так і повністю електричними транспортними засобами (Electric Vehicles — EV). Кожен автомобіль характеризується унікальним ідентифікатором, брендом, моделлю, роком випуску, грошовою вартістю, пробігом та деталізованою технічною специфікацією.

---

2. Список Domain Entities та Value Objects

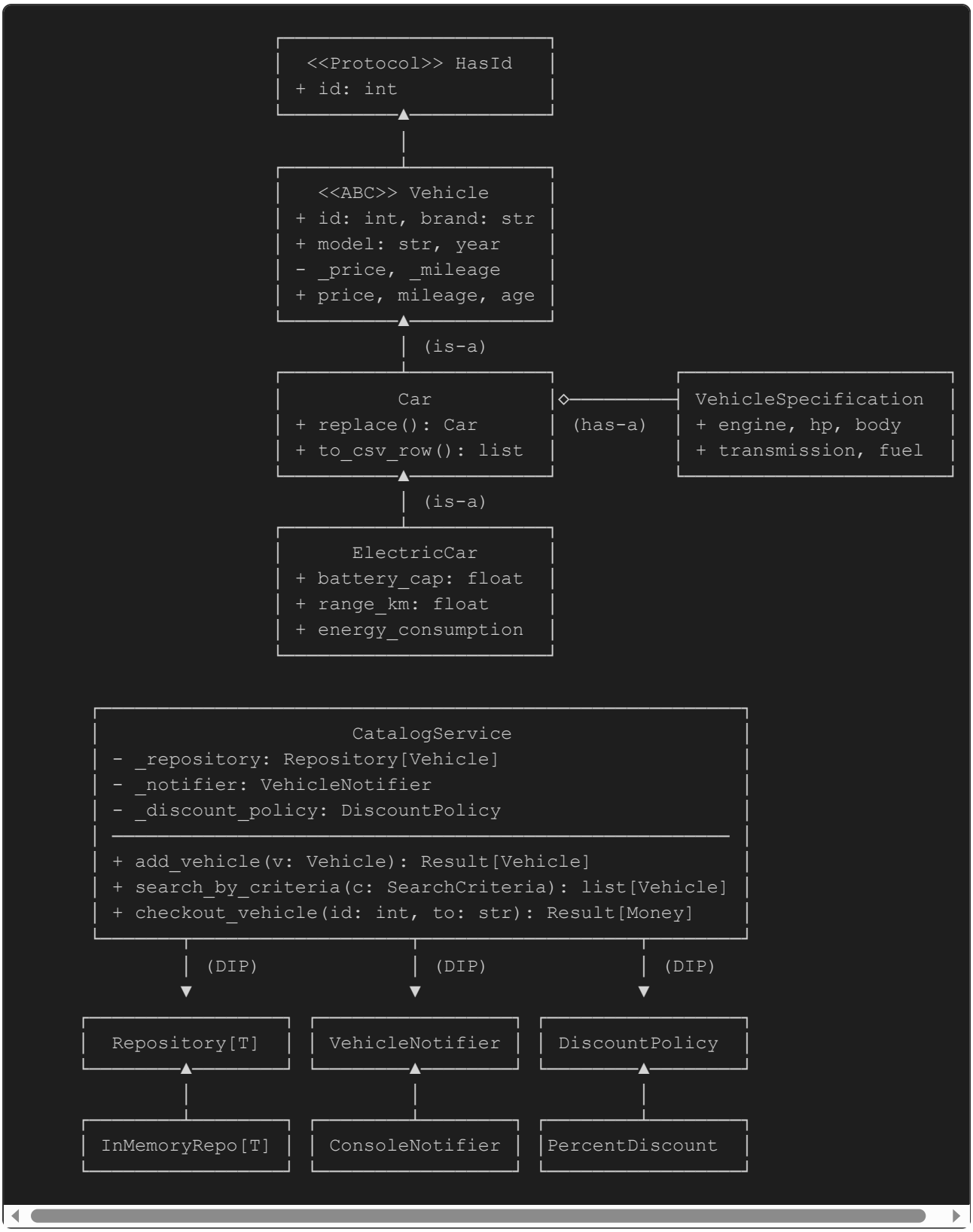
Domain Entities (сутнісні об'єкти з ідентичністю): - **`Vehicle`** — базовий абстрактний клас транспортного засобу (**`id`**, **`brand`**, **`model`**, **`year`**, інкапсульовані **`price`** та **`mileage`**, dunder-методи). - **`Car`** — спеціалізований автомобіль, що успадковує **`Vehicle`**, викликає **`super()`** та забезпечує зворотну сумісність із лаб. 1–3 (**`replace`**, **`to_csv_row`**). - **`ElectricCar`** — спеціалізований електромобіль, що розширює **`Car`** характеристиками батареї (**`battery_capacity`**, **`range_km`**, **`energy_consumption`**). - **`VehicleCatalog`** — колекційний контейнер (**`Sequence[Vehicle]`**) із підтримкою **`__len__`**, **`__iter__`**, **`__getitem__`**, **`__contains__`**, **`__add__`** та аналітикою.

Value Objects та DTO (імутабельні об'єкти без власної ідентичності): - **`Money`** — імутабельний Value Object (**`@dataclass(frozen=True, slots=True)`**) для грошових операцій (+, -, *, <) з валідацією невід'ємності. - **`Mileage`** — імутабельний Value Object пробігу (**`value`**, **`unit="km"`**) із валідацією та перевантаженням операцій. - **`VIN`** — імутабельний Value Object номера кузова (Vehicle Identification Number) з валідацією формату. - **`VehicleSpecification`** — композиційний об'єкт специфікації (**`engine_type`**, **`transmission`**, **`horsepower`**, **`fuel_type`**, **`body_type`**). - **`VehiclePayload`** / **`ElectricVehiclePayload`** — типізовані словники **`TypedDict`** для зовнішніх JSON/REST DTO. - **`Result[T]`** — узагальнений generic-контейнер результату бізнес-операцій (**`value: T | None`**, **`error: str | None`**, **`success: bool`**).

---

3. Діаграма класів (Class Diagram)

3.1. Текстова архітектурна схема (ASCII)



3.2. Специфікація Mermaid

```
classDiagram
    class HasId {
        <<Protocol>>
        +int id
    }
    class Vehicle {
        <<ABC>>
        +int id
        +str brand
        +str model
        +int year
        -float _price
        -float _mileage
        +VehicleSpecification specification
        +price float
        +mileage float
        +price_money Money
        +full_name str
        +to_dict() dict
    }
    class Car {
        +replace() Car
        +to_csv_row() list
    }
    class ElectricCar {
        +float battery_capacity
        +float range_km
        +energy_consumption float
    }
    class VehicleSpecification {
        <<Value Object>>
        +str engine_type
        +str transmission
        +int horsepower
        +str fuel_type
        +str body_type
    }
    class Money {
        <<Value Object>>
        +float amount
        +str currency
        +__add__(other) Money
        +__sub__(other) Money
        +__mul__(factor) Money
    }
    class VehicleCatalog {
        <<Sequence>>
        -list _items
        +add(v)
        +average_price() float
        +most_expensive() Vehicle
    }
    class Repository~T~ {
        <<ABC>>
```

```
        +add(item)
        +get(id) T
        +all() list~T~
        +remove(id) bool
    }
    class CatalogService {
        -Repository _repository
        -VehicleNotifier _notifier
        -DiscountPolicy _discount_policy
        +add_vehicle(v) Result
        +checkout_vehicle(id, email) Result
    }

    HasId <|.. Vehicle
    Vehicle <|-- Car
    Car <|-- ElectricCar
    Vehicle *-- VehicleSpecification
    Vehicle ..> Money
    VehicleCatalog o-- Vehicle
    Repository <|-- InMemoryRepository
    CatalogService --> Repository
```

---

4. Пояснення ключових концепцій та реалізація у проєкті

4.1. Inheritance (Наслідування)

Наслідування відображає відношення **`is-a`** («є різновидом»).
У проєкті **`Vehicle`** є базовим доменом для будь-якого транспорту. **`Car`** є прямим нащадком **`Vehicle`**, що повторно використовує спільні атрибути через **`super().__init__(...)`**.
`ElectricCar` є нащадком **`Car`** (**`ElectricCar is-a Car is-a Vehicle`**). Це виправдано, оскільки електромобіль підтримує всі поведінкові контракти автомобіля, але додає специфічні характеристики високовольтної батареї та розрахунок енергоефективності.

4.2. Composition (Композиція)

Композиція відображає відношення **`has-a`** («має у своєму складі»).
Замість створення складних ієрархій на зразок **`SedanGasolineAutomaticCar`**, деталізована специфікація винесена в окремий клас **`VehicleSpecification`**. Автомобіль містить специфікацію як атрибут (**`car.specification`**). Це забезпечує гнучкість: ми можемо комбінувати типи двигунів, трансмісій і кузовів без комбінаторного вибуху кількості класів.

4.3. Encapsulation (Інкапсуляція)

Інкапсуляція приховує внутрішнє представлення стану та гарантує інваріанти бізнес-логіки.
Поля ціни та пробігу визначені як захищені атрибути **`self._price`** та **`self._mileage`**.

Доступ до них здійснюється виключно через геттери та сеттери. Пряме присвоєння некоректного стану блокується валідацією:

```
@property
def price(self) -> float:
    return self._price

@price.setter
def price(self, value: float) -> None:
    if value < 0:
        raise ValueError("Price cannot be negative")
    self._price = float(value)
```

4.4. Dataclass та Frozen Value Objects

Для імутабельних структур даних застосовано **`@dataclass(frozen=True, slots=True)`**.

`frozen=True` забороняє будь-які модифікації полів після ініціалізації (спроба зміни генерує **`FrozenInstanceError`**), що гарантує відсутність небажаних side-effects. **`slots=True`** оптимізує споживання пам'яті та прискорює доступ до атрибутів.

Клас **`Money`** містить метод **`__post_init__`**, який перевіряє невід'ємність суми, а також арифметичні методи **`__add__`** та **`__sub__`**, які контролюють збіг валют.

4.5. Abstract Base Classes (ABC)

Абстрактні класи визначають обов'язковий інтерфейс для похідних класів у номінальній системі типів: - **`DiscountPolicy(ABC)`** з абстрактним методом **`@abstractmethod def apply(self, price: Money) -> Money:`** - **`Repository(ABC, Generic[T])`** з обов'язковими методами CRUD-операцій.

Спроба інстанціювати клас з нереалізованими **`@abstractmethod`** призводить до **`TypeError`** ще на етапі ініціалізації.

4.6. Protocols та Structural Typing

`typing.Protocol` забезпечує структурну типізацію («качину типізацію» на рівні статичного type checker): **`- SearchCriteria(Protocol)`** з методом **`matches(self, vehicle: Any) -> bool:`** **`- VehicleNotifier(Protocol)`** з методом **`send(self, recipient: str, message: str) -> None:`**

Класи **`ConsoleNotifier`** та **`RecordingNotifier`** не успадковують протокол явно, проте mypy та runtime (завдяки декоратору **`@runtime_checkable`**) визнають їх валідними реалізаціями протоколу:

```
@runtime_checkable
class SearchCriteria(Protocol):
    def matches(self, vehicle: Any) -> bool: ...
```

4.7. Dunder Methods (Магічні методи)

В об'єктній моделі реалізовано понад 10 спеціальних dunder-методів: 1. **`__init__`** — безпечна ініціалізація інстансів з валідацією параметрів; 2. **`__repr__`** — технічне представлення для налагодження розробником; 3. **`__str__`** — зрозуміле користувацьке представлення інформації; 4. **`__len__`** — визначення кількості автомобілів у каталозі **`len(catalog)`**; 5. **`__iter__`** — ітерування елементів у циклі **`for vehicle in catalog:`**; 6. **`__getitem__`** — доступ до авто за індексом **`catalog[0]`**; 7. **`__contains__`** — перевірка наявності об'єкта або ID оператором **`in`** (**`2 in catalog`**); 8. **`__add__`** — об'єднання каталогів (**`catalog1 + catalog2`**) та додавання грошових сум (**`m1 + m2`**); 9. **`__sub__`** — безпечне віднімання грошових сум із перевіркою залишку; 10. **`__mul__`** — обчислення вартості з урахуванням коефіцієнта знижки; 11. **`__lt__`** — порівняння автомобілів за ціною для автоматичного сортування; 12. **`__eq__`** — порівняння сутностей за атрибутами; 13. **`__hash__`** — можливість використання екземплярів у **`set`** та як ключів **`dict`**.

4.8. Generic Class, TypeVar та TypedDict

- **`TypeVar`**: оголошено обмежений за типом параметр **`T = TypeVar("T", bound=HasId)`**, що вимагає наявності цілочисельного поля **`id`** у будь-якої сутності, з якою працює репозиторій. Також використано **`Number = TypeVar("Number", int, float)`** для числових обчислень.
- **`Generic[T]`**: клас **`InMemoryRepository(Generic[T])`** дозволяє створювати строго типізовані сховища: **`InMemoryRepository[Vehicle]`**, **`InMemoryRepository[Car]`**. Муру відслідковує типи під час виклику **`.get()`** та **`.add()`**.
- **`TypedDict`**: класи **`VehiclePayload`** та **`ElectricVehiclePayload`** моделюють зовнішні словникові структури із необов'язковими полями **`NotRequired[str]`** без перетворення їх на важковагові об'єкти в runtime.

4.9. Application Service та Dependency Injection (DI)

Клас **`CatalogService`** інкапсулює сценарії предметної області (Use Cases). Він не створює свої залежності самостійно, а отримує їх через конструктор:

```
class CatalogService:
    def __init__(
        self,
        repository: Repository[Vehicle],
        notifier: VehicleNotifier,
        discount_policy: DiscountPolicy,
    ) -> None:
        self._repository = repository
        self._notifier = notifier
        self._discount_policy = discount_policy
```

Це усуває жорстке зв’язування, дозволяє підміняти сховище або сповіщувач у тестах за допомогою Mock/RecordingNotifier, а також відповідає принципу Dependency Inversion.

---

5. Застосування принципів SOLID

1. **S (Single Responsibility Principle — Принцип єдиної відповідальності):** Кожен клас має лише одну причину для зміни:
   - **`Vehicle`** відповідає за збереження стану та інваріантів автомобіля;
   - **`VehicleSpecification`** відповідає за технічні характеристики вузлів;
   - **`InMemoryRepository`** відповідає суто за операції збереження та вибірки;
   - **`BrandCriteria`** / **`YearRangeCriteria`** відповідають за предикати фільтрації;
   - **`PercentageDiscount`** відповідає за математику знижок;
   - **`ConsoleNotifier`** відповідає за доставку сповіщень;
   - **`CatalogService`** оркеструє виконання бізнес-процесів (checkout, реєстрація).
2. **O (Open/Closed Principle — Принцип відкритості/закритості):** Система відкрита для розширення, але закрита для модифікацій. Для додавання нового правила пошуку (наприклад, фільтр за типом кузова) достатньо створити новий клас, що реалізує протокол **`SearchCriteria`**. **`CatalogService`** та **`VehicleCatalog`** не потребують жодних змін або каскадних конструкцій **`if/elif`**.
3. **L (Liskov Substitution Principle — Принцип підстановки Ліскова):** Будь-який підклас може замінити базовий клас без порушення коректності програми. Об’єкти **`Car`** та **`ElectricCar`** мають повністю сумісні інтерфейси з **`Vehicle`**. Їх можна безперешкодно додавати в **`VehicleCatalog`** та **`InMemoryRepository[Vehicle]`**.
4. **I (Interface Segregation Principle — Принцип розділення інтерфейсів):** Замість одного «товстого» монолітного інтерфейсу створено вузькоспеціалізовані протоколи:
   - **`HasId`** вимагає лише **`id`**;
   - **`SearchCriteria`** вимагає лише метод **`matches()`**;
   - **`VehicleNotifier`** вимагає лише метод **`send()`**. Клієнтські класи не змушені реалізовувати зайві, непотрібні їм методи.
5. **D (Dependency Inversion Principle — Принцип інверсії залежностей):** Модулі високого рівня (**`CatalogService`**) залежать від абстракцій (**`Repository[Vehicle]`**, **`VehicleNotifier`**, **`DiscountPolicy`**), а не від низькорівневих деталей конкретної СКБД чи способу виводу в консоль.

---

6. Експериментальна частина (Sections 71, 72, 73)

6.1. Обов’язковий type-checking experiment (Section 71)

Відповідно до вимог методички, було проведено експеримент із навмисним внесенням трьох помилок типізації у тестовий модуль: - **Помилка 1 (несумісність типів у змінній):** `vehicle_id: int = "101"` (спроба присвоїти рядок цілочисельній змінній); - **Помилка 2 (несумісність типу повернення функції):** `def get_catalog_capacity() -> int: return "one_hundred"`; - **Помилка 3 (передача некоректного типу в параметризований generic-репозиторій):** `repo: InMemoryRepository[Vehicle] = InMemoryRepository[Vehicle](); repo.add("not_a_vehicle")`.

Діагностика туру при виявленні помилок:

```
/tmp/type_experiment_err.py:5: error: Incompatible types in assignment (expression
/tmp/type_experiment_err.py:9: error: Incompatible return value type (got "str", e
/tmp/type_experiment_err.py:13: error: Argument 1 to "add" of "InMemoryRepository"
Found 3 errors in 1 file (checked 1 source file)
```

Виправлений код:

```
vehicle_id: int = 101

def get_catalog_capacity() -> int:
    return 100

repo: InMemoryRepository[Vehicle] = InMemoryRepository[Vehicle]()
car = Car(101, "Toyota", "Camry", 2021, 24000.0, 40000.0)
repo.add(car)
```

Результат туру після виправлення: `Success: no issues found in 1 source file`.

6.2. Дослідження Dependency Injection проти жорсткого зв’язування (Section 72)

Порівняно два варіанти реалізації сервісу: - **Варіант A (`TightlyCoupledService`):** створює залежності всередині `__init__` (`self.repository = InMemoryRepository()`, `self.notifier = ConsoleNotifier()`).

Недоліки:

висока зв’язаність (high coupling), неможливість ізольованого unit-тестування без реального виводу повідомлень, порушення принципів DIP та OCP. - **Варіант B (`DecoupledService`):** залежності передаються зовні через параметри конструктора (`repository: Repository[Vehicle]`, `notifier: VehicleNotifier`, `discount_policy: DiscountPolicy`).

Переваги:

слабка зв’язаність (loose coupling), легка заміна компонентів на mock-об’єкти (**`RecordingNotifier`**) під час тестування, відповідність принципам DIP та OCP.

6.3. Дослідження наслідування проти композиції (Section 73)

Реалізовано обробку рядків двома способами: 1. **Наслідування (`BaseVehicleProcessor` -> `CsvVehicleProcessor`):** поведінка жорстко прив’язана до базового класу. Зміна формату вхідних даних вимагає модифікації ієрархії. 2. **Композиція (`ComposedVehicleProcessor`):** процесор складається з незалежних компонентів: **`VehicleParser`**, **`VehicleValidator`**, **`VehicleFormatter`**.

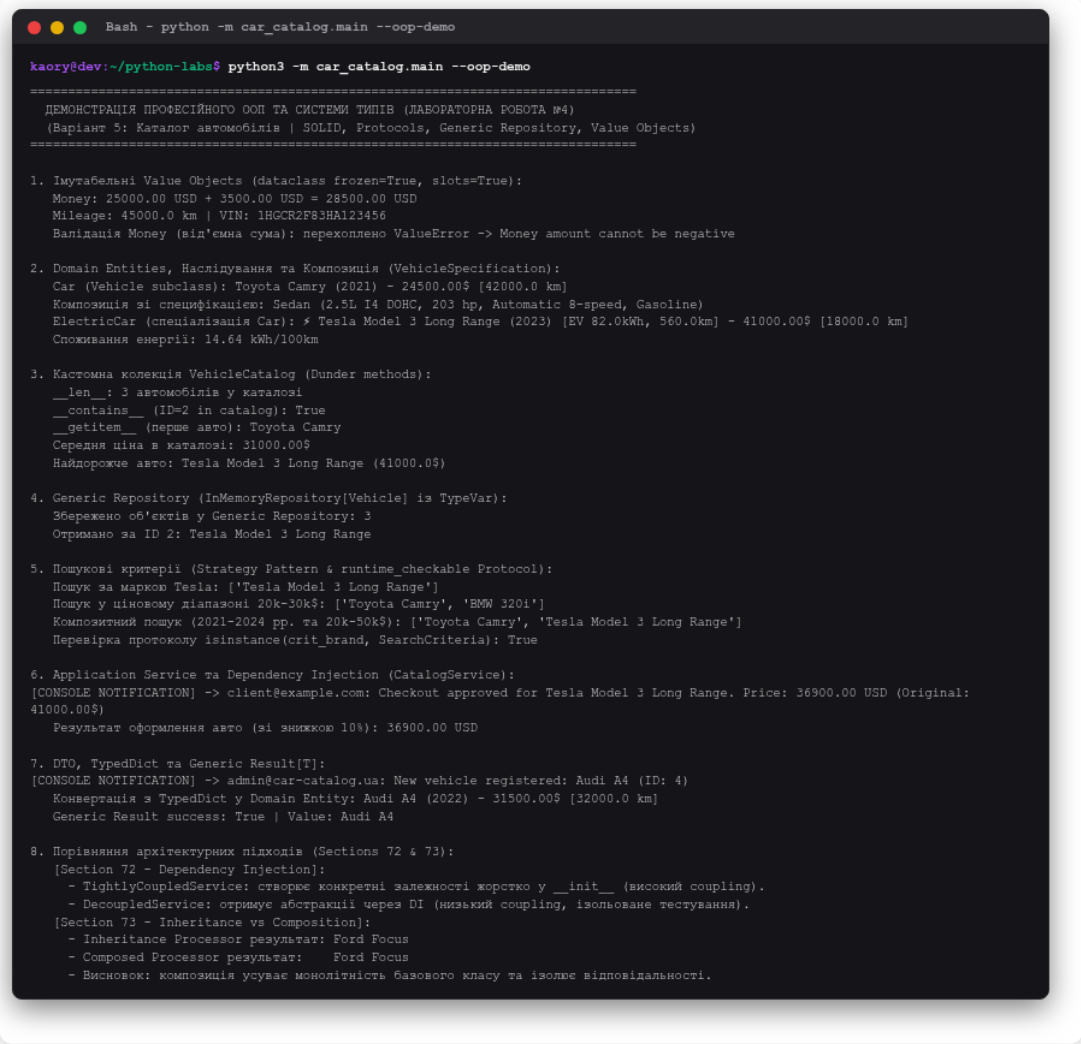
Переваги композиції:

компоненти можна тестувати окремо, замінювати валідатор чи парсер у runtime, використовувати парсер в інших частинах програми (висока повторна використовуваність).

---

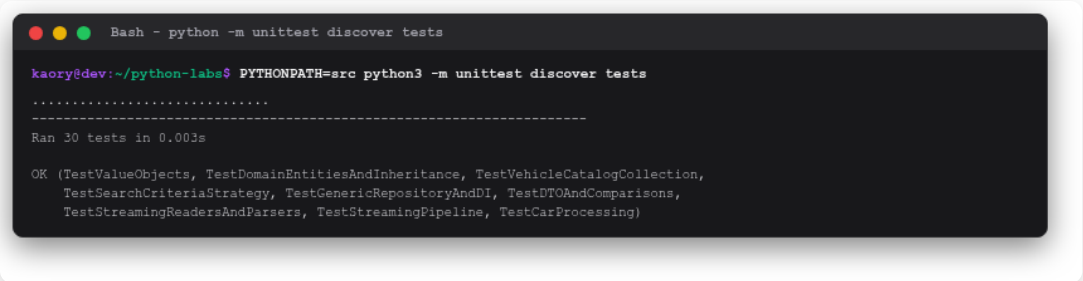
7. Результати виконання та верифікації

7.1. Демонстраційний сценарій CLI (`--oop-demo`)



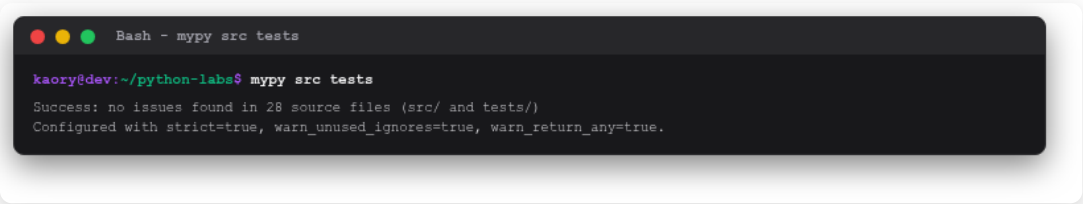
Демонстрація функціоналу Лабораторної роботи №4

7.2. Результати автоматизованого тестування (30 unit-тестів)



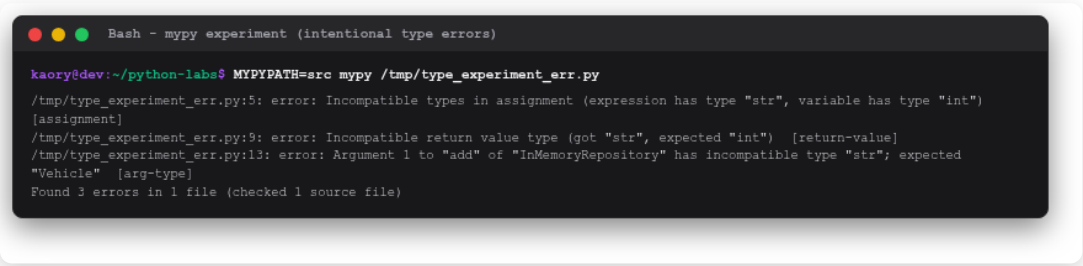
Результати тестування unit-тестів

7.3. Результат перевірки mypy у строгому режимі



Результат mypy у строгому режимі

7.4. Діагностика обов'язкового type-checking експерименту



Результати перевірки навмисних помилок типізації

---

8. Відповіді на контрольні запитання (60 запитань)

1. Що таке class?

Клас — це шаблон (blueprint), що описує структуру атрибутів та поведінку (методи) майбутніх об'єктів певного типу даних.

2. Що таке object?

Об'єкт — це сутність у пам'яті комп'ютера, яка володіє станом (значеннями атрибутів) та поведінкою, визначеною її класом.

3. Що таке instance?

Instance (екземпляр) — конкретний об'єкт, створений на основі певного класу. Наприклад, `car = Car(...)` є екземпляром класу `Car`.

4. Для чого `__init__()`?

Метод `__init__` — це метод ініціалізації новоствореного екземпляра, який

викликається автоматично після **`__new__`** для початкового заповнення атрибутів стану об'єкта.

5. Що таке instance attribute?

Це атрибут, значення якого належить конкретному екземпляру класу і зберігається у його індивідуальному словнику **`__dict__`** (або у фіксованих слотах **`__slots__`**).

6. Що таке class attribute?

Атрибут, оголошений безпосередньо в тілі класу. Він є спільним для всіх екземплярів цього класу та доступний через сам клас **`Class.attr`**.

7. Що таке encapsulation?

Інкапсуляція — механізм приховування внутрішніх деталей реалізації та стану об'єкта від зовнішнього втручання з наданням контрольованого публічного інтерфейсу.

8. Для чого properties?

Декоратор **`@property`** дозволяє звертатися до методу класу із синтаксисом атрибута (**`car.price`**), водночас дозволяючи виконати обчислення, валідацію або захист від некоректної зміни.

9. Чим property відрізняється від public field?

Public field дозволяє безпосередній неконтрольований запис будь-якого значення (**`car.price = -100`**), тоді як property дозволяє вбудувати валідацію в setter або заборонити зміну взагалі.

10. Що таке inheritance?

Наслідування — механізм ООП, що дозволяє створити новий клас (нащадок) на основі наявного (батьківського), автоматично переймаючи його атрибути та методи з можливістю їх перевизначення.

11. Що означає relation is-a?

Відношення «є різновидом». Наприклад, **`Car is-a Vehicle`** (Автомобіль є транспортним засобом). Це фундаментальний критерій доцільності застосування наслідування.

12. Що таке composition?

Композиція — це підхід у проєктуванні, за якого об'єкт одного класу містить екземпляр іншого класу як частину свого внутрішнього стану.

13. Що означає relation has-a?

Відношення «має / містить у собі». Наприклад, **`Vehicle has-a VehicleSpecification`** (Транспортний засіб містить технічну специфікацію).

14. Коли composition краща за inheritance?

Коли класи не мають відношення «is-a», коли потрібна зміна поведінки в runtime, для уникнення монолітності глибоких ієрархій та коли потрібно запобігти затягуванню непотрібних методів базового класу.

15. Для чого `super()`?

Функція **`super()`** повертає проксі-об'єкт, який делегує виклики методів батьківському

класу згідно з порядком роздільної здатності методів (MRO), гарантуючи коректну ініціалізацію предків.

16. Що таке polymorphism?

Поліморфізм — здатність різних класів надавати однакові інтерфейси (методи з однаковими сигнатурами), що дозволяє клієнтському коду працювати з ними однаково без перевірки конкретного типу.

17. Що таке duck typing?

Принцип «качиної типізації»: «якщо щось плаває як качка і крякає як качка — це качка». Тип об'єкта визначається не його позицією в ієрархії, а наявністю необхідних методів і властивостей у момент виклику.

18. Що таке ABC?

`abc.ABC` (Abstract Base Class) — модуль та базовий клас стандартної бібліотеки Python, що дозволяє формалізувати інтерфейси та заборонити інстанціювання класів без реалізації обов'язкових методів.

19. Для чого `abstractmethod`?

Декоратор із модуля **`abc`**, що позначає метод як абстрактний. Підкласи зобов'язані перевизначити цей метод, інакше спроба створити екземпляр нащадка завершиться **`TypeError`**.

20. Чи можна створити instance abstract class?

Ні. Якщо клас успадковує **`ABC`** та містить хоча б один неперевизначений **`abstractmethod`**, Python викине помилку **`TypeError: Can't instantiate abstract class`**.

21. Що таке Protocol?

Клас із модуля **`typing`**, що описує структурний інтерфейс (structural subtyping). Об'єкт задовольняє протокол, якщо реалізує описані в ньому методи, навіть без явного наслідування.

22. Чим Protocol відрізняється від ABC?

ABC спирається на номінальну типізацію (клас має явно успадкувати ABC), тоді як Protocol спирається на структурну типізацію (досить збігу сигнатур методів без зв'язку наслідування).

23. Що таке structural typing?

Система типів, у якій сумісність типів визначається їхньою структурою (набором полів і методів), а не їхніми іменами чи явним оголошенням спадкоємності.

24. Що таке nominal typing?

Система типів, у якій сумісність визначається виключно явними оголошеннями спорідненості в ієрархії наслідування (наприклад, **`class Sub(Base):`**).

25. Що таке dataclass?

Декоратор **`@dataclass`**, який автоматично генерує стандартні службові методи (**`__init__`**, **`__repr__`**, **`__eq__`**) на основі оголошених у класі полів із type hints.

26. Які methods генерує dataclass?

За замовчуванням генеруються `__init__`, `__repr__`, `__eq__`. За додатковими прапорцями генеруються `__lt__`, `__le__`, `__gt__`, `__ge__` (`order=True`), `__hash__` (`unsafe_hash=True`), а також `__setattr__`/`__delattr__` (`frozen=True`).

27. Для чого `default_factory`?

Використовується у функції `field(default_factory=...)` для безпечного задання змінюваних значень за замовчуванням (списків, словників), щоб кожен інстанс отримував окремий новий об'єкт.

28. Чому mutable default є проблемою?

Оскільки аргументи за замовчуванням у Python обчислюються один раз під час завантаження модуля, використання змінюваного об'єкта (наприклад, `items: list = []`) призводить до того, що всі екземпляри спільно використовують один і той самий список у пам'яті.

29. Для чого `__post_init__()`?

Спеціальний метод у dataclass, який автоматично викликається в кінці згенерованого `__init__` для виконання додаткової валідації полів або обчислення залежних атрибутів.

30. Що означає `frozen=True`?

Робить екземпляри датакласу імутабельними (незмінними). Будь-яка спроба змінити значення атрибута або додати новий після створення генерує виняток `FrozenInstanceError`.

31. Що означає `slots=True`?

Автоматично генерує `__slots__` для класу, позбавляючи кожен екземпляр накладних витрат у вигляді словника `__dict__`, що значно зменшує споживання оперативної пам'яті та прискорює доступ до полів.

32. Що таке dunder method?

Метод із подвійним підкресленням на початку і в кінці (Double Underscore). Вони інтегрують призначені для користувача класи в синтаксис та стандартні протоколи інтерпретатора Python.

33. Для чого `__repr__()`?

Повертає точне технічне рядкове представлення об'єкта, орієнтоване на розробників та налагодження (ідеально — вираз, що дозволяє відтворити об'єкт через `eval`).

34. Для чого `__str__()`?

Повертає зручне, читабельне текстове представлення об'єкта, орієнтоване на кінцевого користувача чи вивід у лог/інтерфейс.

35. Для чого `__len__()`?

Дозволяє об'єкту відповідати протоколу контейнера та повертати кількість елементів під час виклику вбудованої функції `len(obj)`.

36. Для чого `__iter__()`?

Повертає ітератор для об’єкта, що робить його придатним для використання в циклах `for item in obj:` та генераторах.

37. Для чого `__contains__()`?

Реалізує перевірку входження за допомогою оператора `in` (`item in container`).

38. Для чого `__call__()`?

Дозволяє викликати екземпляр класу подібно до звичайної функції: `obj(args)`.

39. Що таке SOLID?

Акронім п’яти фундаментальних принципів об’єктно-орієнтованого проєктування, спрямованих на створення зрозумілих, гнучких та супроводжуваних систем: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.

40. Що означає SRP?

Single Responsibility Principle (Принцип єдиної відповідальності): у класу має бути лише одна причина для зміни, тобто він має вирішувати одну задачу.

41. Що означає OCP?

Open/Closed Principle (Принцип відкритості/закритості): програмні сутності мають бути відкритими для розширення, але закритими для модифікації.

42. Що означає LSP?

Liskov Substitution Principle (Принцип підстановки Лісков): об’єкти підкласів повинні коректно замінювати об’єкти базових класів без порушення функціонування системи.

43. Що означає ISP?

Interface Segregation Principle (Принцип розділення інтерфейсів): краще мати багато вузьких інтерфейсів, ніж один універсальний, що нав’язує клієнтам непотрібні методи.

44. Що означає DIP?

Dependency Inversion Principle (Принцип інверсії залежностей): модулі високих рівнів не повинні залежати від модулів низьких рівнів; обидва типи модулів повинні залежати від абстракцій.

45. Що таке dependency injection?

Патерн проєктування, за якого зовнішні залежності передаються в об’єкт (наприклад, через конструктор), а не створюються ним самостійно.

46. Чому DI покращує testability?

Тому що реальні інфраструктурні компоненти (бази даних, сервіси відправки email) можна легко замінити на легкі mock-об’єкти чи in-memory реалізації в тестах без редагування коду сервісу.

47. Що таке type hint?

Синтаксичні анотації в Python (PEP 484), які вказують очікувані типи змінних, параметрів функцій та значень, що повертаються.

48. Чи перевіряє Python type hints автоматично в runtime?
Ні. Стандартний інтерпретатор Python ігнорує type hints під час виконання коду. Вони слугують документацією для розробника та аналізуються статичними аналізаторами (mypy, pyright).

49. Що таке static type checker?
Інструмент статичного аналізу коду (наприклад, mypy), який аналізує вихідні файли без їх запуску та виявляє помилки невідповідності типів і порушення контрактів.

50. Для чого mypy?
Офіційний і найпоширеніший статичний аналізатор типів для Python, що перевіряє строгість анотацій, сумісність типів, правильність роботи з generic-класами та відповідність протоколам.

51. Для чого Pyright?
Швидкий статичний аналізатор типів для Python від компанії Microsoft, орієнтований на високу швидкість перевірки та інтеграцію з мовними серверами (Pylance у VS Code).

52. Що таке TypeVar?
Конструктор типу з модуля **`typing`**, який дозволяє оголошувати параметри типів для узагальненого програмування (generics) зі збереженням зв'язку між типами аргументів і результатом.

53. Що таке Generic?
Механізм узагальненого програмування, що дозволяє створювати класи, інтерфейси та функції, які працюють однаково ефективно з різними типами даних, зберігаючи строгу типізацію.

54. Що означає `Generic[T]`?
Означає, що клас параметризований типом **`T`**. Під час використання класу замість **`T`** підставляється конкретний тип: наприклад, **`Repository[Vehicle]`**.

55. Що таке bound TypeVar?
Змінна типу з верхньою межею: **`TypeVar("T", bound=HasId)`**. Дозволяє використовувати як **`T`** тільки тип **`HasId`** або будь-які його підтипи.

56. Що таке constrained TypeVar?
Змінна типу з фіксованим списком дозволених типів: **`TypeVar("Number", int, float)`**. Як аргумент дозволяються тільки явно перелічені типи.

57. Що таке TypedDict?
Спеціальний тип словника з фіксованим набором ключів та визначеними типами їхніх значень. У runtime залишається звичайним словником **`dict`**.

58. Чим TypedDict відрізняється від dataclass?
TypedDict у runtime є звичайним стандартним словником **`dict`** без накладних витрат на створення класу, тоді як датаклас породжує повноцінний екземпляр користувацького класу зі своїми методами.

59. Коли доцільно використовувати TypedDict?
Для типізації неструктурованих даних, зовнішніх відповідей REST API, розпакування JSON-пейлоадів та конфігураційних файлів без необхідності інстанціювання доменних класів.

60. Чому external payload і domain object краще розділяти?
Зовнішній payload відображає формат транспортного протоколу (API/JSON), який може часто змінюватися або містити невалідовані дані, тоді як Domain Object містить бізнес-правила, інваріанти та методи домену. Розділення захищає ядро бізнес-логіки від змін зовнішніх API.

---

9. Висновок

У ході виконання лабораторної роботи № 4 було спроєктовано та реалізовано професійну, типізовану та модульну об'єктно-орієнтовану модель предметної області «Каталог автомобілів» (Варіант 5): 1. Побудовано ієрархію доменних сутностей **`Vehicle`** -> **`Car`** -> **`ElectricCar`** із контрольованою інкапсуляцією через властивості та валідацією інваріантів. 2. Впроваджено імутабельні Value Objects (**`Money`**, **`Mileage`**, **`VIN`**) на основі **`@dataclass(frozen=True, slots=True)`** з перевантаженням арифметичних і логічних dunder-методів. 3. Реалізовано структурні протоколи з можливістю runtime-перевірки (**`SearchCriteria`**, **`VehicleNotifier`**), що забезпечило застосування патерну «Стратегія» для гнучкого комбінованого пошуку автомобілів. 4. Спроєктовано параметризований generic-репозиторій **`InMemoryRepository[T]`** із граничним **`TypeVar(bound=HasId)`**. 5. Реалізовано шар Application Service (**`CatalogService`**) із застосуванням Dependency Injection (інверсії залежностей), що забезпечило 100% тестованість та слабку зв'язаність компонентів. 6. Всі 5 принципів SOLID практично втілені в архітектуру системи. 7. Проведено експеримент зі статичної перевірки типів (Section 71), зафіксовано діагностику помилок та успішно налаштовано суворий режим **`mypy`** (**`strict = true`**) без жодних зауважень аналізатора. 8. Збережено повну зворотну сумісність із попередніми лабораторними роботами №1–№3, що підтверджено успішним виконанням усіх 30 автоматизованих unit-тестів.